Magic Methods

1. Create a class called `Point`
2. Add an `__init__` method with `x` and `y`, and store them as properties
3. Add a `__repr__` method that returns `f"Point({x}, {y})"`
4. Create an object `p1` with x **2** and y **3**
5. Print `p1`

Inheritance

1. Create a parent class `Animal` with an `__init__` that takes `name`
2. Add a method `speak` that prints the name
3. Create a child class `Dog` that inherits from `Animal`
4. Create an object `d1 = Dog("Rex")`
5. Call `d1.speak()`

`Polymorphism`

1. Create a class `Cat` with a method `sound` that prints **"Meow"**
2. Create a class `Fox` with a method `sound` that prints **"Wa-pa-pa-pa-pa-pow!"**
3. Create objects `c1 = Cat()` and `f1 = Fox()`
4. Call `sound()` on both objects

Encapsulation

1. Create a class `ScoreBoard`
2. Add an `__init__` with a `score` parameter and store it as a private attribute
3. Add a method called `get_score` that returns the private score
4. Create an object `s1` with a score of **0**
5. Print the score of `s1`